COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

ARTIFICIAL INTELLIGENCE: **LAB ASSIGNMENT**

Name:MOHIZ KHAN: **REG NO:FA24-BCS-013**

2) Solved Lab Activities :

Activity 1:

The BFS search starts from a node, then it checks all the nodes at distance one from the beginning node, then it checks all the nodes at distance two, and so on. So as to recollect the nodes to be visited, BFS uses a queue.

The steps of the algorithm work as follow:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

Figure: Graph for BFS Implementation

Many times, a graph may contain two different disconnected parts and therefore to make sure that we have visited every vertex, we can also run the BFS algorithm at every node.

[2]
0s

```python
# Graph definition
graph = {
    '5': ['3','7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

from collections import deque

def bfs(graph, start):
    visited = []
    queue = deque([start])

    visited.append(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

print("Following is the Breadth-First Search:")
bfs(graph, '5')
```

```
Following is the Breadth-First Search:
5 3 7 2 4 8
```

Activity 2:

Try implementing BFS for a tree now, using the following tree structure: Figure: Tree for BFS Implementation The code to make the tree is given as # python program to demonstrate some of the above # terminologies # Function to add an edge between vertices x and y # Function to print the parent of each node

```
[3]
✓ 0s
# Functions

def printParents(node, adj, parent):
    if parent == 0:
        print(node, "-> Root")
    else:
        print(node, "->", parent)

    for cur in adj[node]:
        if cur != parent:
            printParents(cur, adj, node)


def printChildren(root, adj):
    from collections import deque
    q = deque([root])
    vis = [0] * len(adj)

    while q:
        node = q.popleft()
        vis[node] = 1

        print(node, "-> ", end="")

        for cur in adj[node]:
            if vis[cur] == 0:
                print(cur, end=" ")
                q.append(cur)
        print()


def printLeafNodes(root, adj):
    for i in range(1, len(adj)):
        if len(adj[i]) == 1 and i != root:
            print(i, end=" ")
    print()


def printDegrees(root, adj):
    for i in range(1, len(adj)):
        if i == root:
            print(i, ":", len(adj[i]))
        else:
            print(i, ":", len(adj[i]) - 1)


# Driver Code
N = 7
```

```python
# Driver Code
N = 7
root = 1

adj = [[] for _ in range(N + 1)]

# Tree edges
adj[1].extend([2, 3, 4])
adj[2].extend([1, 5, 6])
adj[3].append(1)
adj[4].extend([1, 7])
adj[5].append(2)
adj[6].append(2)
adj[7].append(4)

print("Parents:")
printParents(root, adj, 0)

print("\nChildren:")
printChildren(root, adj)

print("\nLeaf Nodes:")
printLeafNodes(root, adj)

print("\nDegrees:")
printDegrees(root, adj)
```

```
Parents:
1 -> Root
2 -> 1
5 -> 2
6 -> 2
3 -> 1
4 -> 1
7 -> 4

Children:
1 -> 2 3 4
2 -> 5 6
3 ->
4 -> 7
5 ->
6 ->
7 ->

Leaf Nodes:
3 5 6 7

Degrees:
1 : 3
2 : 2
3 : 0
4 : 1
5 : 0
6 : 0
```

Activity 3: Using the code in ‘activity 1’ apply the same procedure to the graph give in the Figure below:
Graph for Activity :3

```python
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

# Example graph (replace with your figure)
graph = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['E'],
    'D': [],
    'E': []
}

bfs(graph, 'A')
```

```
A B C D E
```

Activity 4:

Lab task 01:

Imagine the same grapch we discussed above but this time we also mention the cost of each edge. Figure: Weighted Graph for Activity Lab Task 1 First we modify our graph structure so that in Node.actions is an array of tuples where each tuple contains a vertex and its associated weight.

```python
from collections import deque

graph = {
    'A': [('B', 2), ('C', 1)],
    'B': [('D', 5)],
    'C': [('D', 3)],
    'D': [('G', 2)],
    'G': []
}

def bfs_cost(graph, start, goal):
    queue = deque([(start, 0)])
    visited = set()

    while queue:
        node, cost = queue.popleft()

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Reached Goal:", node)
            print("Total Cost:", cost)
            return

        for neighbour, weight in graph[node]:
            queue.append((neighbour, cost + weight))

bfs_cost(graph, 'A', 'G')
```

```
Reached Goal: G
Total Cost: 9
```

Lab Task 2:

In order to move onto more complex implementations of BFS let us consider a concise version of Travelling Salesman Problem given in Figure below: Figure: Graph for Travelling Salesman Problem Figure: Travelling Salesman Problem i. Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node) ii. Apply breadth first search to reach the goal state, your output should have the path and the total distance to reach the path. iii. Save your code and results as BFS_REG#.py as it would be used in later exercises.

[7]
0s

```python
from collections import deque

graph = {
    'Arad': [('Sibiu', 140), ('Timisoara', 118), ('Zerind', 75)],
    'Sibiu': [('Fagaras', 99), ('Rimnicu', 80)],
    'Fagaras': [('Bucharest', 211)],
    'Rimnicu': [('Pitesti', 97)],
    'Pitesti': [('Bucharest', 101)],
    'Timisoara': [],
    'Zerind': [],
    'Bucharest': []
}

def bfs_path(graph, start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()

    while queue:
        node, path, cost = queue.popleft()

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            print("Path:", " -> ".join(path))
            print("Total Distance:", cost)
            return

        for neighbour, weight in graph[node]:
            queue.append((neighbour, path + [neighbour], cost + weight))

bfs_path(graph, 'Arad', 'Bucharest')
```

```
Path: Arad -> Sibiu -> Fagaras -> Bucharest
Total Distance: 450
```